

Agence financière immobilière — Mini-projet

Objectif

En tant que développeur ou développeuse, vous devez analyser les besoins d'un client, proposer une première structure de composants, puis réaliser une interface React fonctionnelle, claire, réutilisable et maintenable à partir d'un code de départ fourni par l'enseignante.

Pondération / remise

Élément	Information
Pondération	25 % de la note finale
Partie 1	Travail en classe + dépôt de progression obligatoire; les éléments d'analyse peuvent être améliorés jusqu'à la remise finale.
Partie 2	Remise finale du dépôt GitHub le 30 septembre.
Partie 3	Entrevue individuelle après la remise finale; la séance et l'heure seront annoncées en classe.

Notions et objets d'évaluation

- Analyser un besoin et formuler clairement ce que l'application doit permettre de faire.
- Décomposer une interface React en composants ayant des responsabilités claires.
- Utiliser React-Bootstrap et des CSS Modules pour construire une interface cohérente et responsive.
- Utiliser useState, les événements, les props et l'affichage conditionnel.
- Afficher et modifier des listes avec map(), key, filter() et les setters de state.
- Organiser le code afin d'éviter les répétitions inutiles et favoriser la réutilisation.
- Utiliser GitHub pour conserver la progression et remettre le travail.
- Expliquer et justifier ses choix lors d'une courte entrevue individuelle.

Mise en situation — cahier des charges du mini-projet

Vous êtes mandaté(e) par une agence financière immobilière fictive. L'agence acquiert des terrains, participe au financement et au développement de projets immobiliers, puis met en marché et revend des propriétés résidentielles ou commerciales. Elle souhaite disposer d'une interface Web simple qui présente ses activités et ses projets de manière claire et professionnelle.

Votre rôle n'est pas seulement de « coder ce qui est écrit ». Vous devez d'abord analyser la demande, identifier les besoins du site, puis proposer une structure adaptée avant de poursuivre le développement.

Ce que le site doit permettre de faire

- Présenter l'agence et donner une vue d'ensemble de ses activités.
- Permettre de naviguer entre au minimum les sections suivantes : Accueil, Projets, Services, À propos et Nous joindre.
- Présenter une liste de projets immobiliers et permettre de consulter les informations essentielles de chaque projet.
- Permettre de filtrer les projets selon un critère pertinent, par exemple le type de projet ou son statut.
- Permettre de retirer un projet de la liste affichée à partir de l'interface.
- Indiquer clairement lorsqu'aucun projet ne correspond au filtre choisi.
- Donner un retour visuel à l'utilisateur afin qu'il sache quelle section ou quel filtre est actif.
- Rester lisible, cohérent et utilisable sur différentes tailles d'écran.

Programmation avancée

Données minimales pour les projets

La version finale doit contenir au moins 12 projets immobiliers. Pour chaque projet, les informations minimales suivantes doivent être disponibles :

- un identifiant unique;
- un titre;
- une ville ou un secteur;
- un type;
- un statut;
- une courte description;
- une information financière ou une information de superficie.
- Une image;

Contraintes du projet

- Le TP porte sur l'interface graphique et les notions React vues dans le cours.
- Aucun backend, base de données, API, authentification ou React Router n'est demandé.
- Le projet de départ fourni par l'enseignante doit être utilisé. Il n'est pas permis de remplacer le projet par une autre application ni de recommencer entièrement à zéro.
- Les bibliothèques ou fonctionnalités non vues en classe ne sont pas nécessaires pour obtenir tous les points.

Description du travail à faire

Partie 1 — Analyse, structure et prise en main du code (5 %)

Modalité : cette partie se réalise en classe, sous supervision. Elle sert à produire une première analyse, une première organisation de l'application et une trace de démarrage personnel.

Important : la note de cette partie sera attribuée à partir de la version finale de votre analyse et de votre structure. Le dépôt effectué à la fin de la séance sert de point de contrôle et de preuve de progression. Vous pourrez donc compléter ou corriger votre analyse et votre arbre des composants avant la remise finale.

Étape A — Analyser les besoins

À partir du cahier des charges, rédigez une liste claire des besoins fonctionnels et techniques de l'application. Formulez-les du point de vue de ce que le site doit permettre à l'utilisateur de faire, et non du point de vue du code à écrire.

Exemple de formulation attendue : « Le site doit permettre à l'utilisateur de filtrer les projets affichés selon leur type. »

- Identifier les fonctionnalités obligatoires.
- Identifier les informations que le site doit afficher.
- Identifier les comportements attendus lorsque l'utilisateur interagit avec l'interface.
- Identifier les contraintes de qualité pertinentes : clarté, cohérence visuelle, responsive, facilité d'utilisation.

Étape B — Proposer une première version de l'arbre des composants

À partir de votre analyse, proposez une première décomposition de l'interface en composants React. Votre arbre doit montrer les relations parent-enfant et permettre de comprendre la responsabilité principale de chaque composant.

- Nommer les composants de façon claire et cohérente.
- Éviter de créer un composant sans responsabilité précise.
- Repérer les éléments qui peuvent être réutilisés au lieu d'être copiés plusieurs fois.
- Prévoir que l'arbre pourra évoluer lorsque vous découvrirez le code fourni et pendant le développement.

Étape C — Recevoir et corriger le code de départ

Le projet React de départ : <https://github.com/HadiaMCH/Agence-immobiliere-react>

Ce projet contient volontairement des erreurs. Vous devez les repérer et les corriger afin d'obtenir une base de travail fonctionnelle.

- Utiliser le projet fourni tel qu'il est remis comme point de départ.
- Faire un peu de ménage initiale.
- Corriger et documenter les erreurs empêchant le projet de fonctionner correctement.
- Ne pas refaire l'application à zéro.
- Conserver une structure de projet compréhensible et préparer le code pour la suite du TP.

Programmation avancée

Étape D — Dépôt de progression obligatoire

- Avant de quitter la séance, effectuer au moins un commit significatif et pousser le travail sur votre dépôt GitHub.
- Le dépôt doit contenir la progression réellement effectuée pendant la séance.
- Après cette première remise de progression, vous pouvez continuer à améliorer l'analyse, l'arbre des composants et le code jusqu'à la remise finale.

Critère — Partie 1	Ce qui sera évalué dans la version finale	Points
Analyse des besoins	Besoins clairs, complets et formulés en fonction de ce que le site doit permettre de faire.	2,0
Arbre des composants	Structure logique, relations parent-enfant cohérentes et responsabilités compréhensibles.	1,5
Prise en main du code fourni	Erreurs de départ corrigées et projet utilisé comme base réelle du développement.	1,0
Trace GitHub de la séance	Commit et push de progression présents dans l'historique.	0,5

Partie 2 — Livrable final : solution fonctionnelle et bien structurée (10 %)

Modalité : cette partie est réalisée individuellement à partir du même dépôt GitHub commencé en classe.

Objectif : remettre une solution complète qui répond au cahier des charges et qui démontre une organisation de code claire, cohérente et réutilisable.

A. Fonctionnement attendu

- Toutes les fonctionnalités obligatoires décrites dans le cahier des charges doivent être présentes et fonctionner.
- La navigation entre les sections doit fonctionner par affichage conditionnel.
- La section Projets doit afficher au moins 8 projets à partir de données conservées dans un state.
- Les projets doivent être générés avec `map()` et utiliser une key stable basée sur leur id.
- Un système de filtre doit utiliser `filter()` et un state représentant le filtre actif.
- Le filtre actif doit être identifiable visuellement.
- Chaque projet doit être affiché au moyen d'un composant de carte réutilisable.
- Le composant de carte doit recevoir ses données par props et recevoir au moins une fonction du parent par prop.
- Un bouton doit permettre de retirer un projet de la liste en créant une nouvelle liste avec `filter()`, puis en utilisant le setter du state.
- Un message conditionnel doit être affiché lorsqu'aucun projet ne correspond au filtre choisi.

B. Qualité et organisation du code

- Le projet doit démarrer et fonctionner sans erreur de compilation.
- Le code doit être organisé dans une structure de dossiers et de fichiers facile à comprendre.
- Chaque composant doit avoir une responsabilité claire et un nom représentatif de son rôle.
- Les composants réutilisables doivent être réellement réutilisés; évitez de copier plusieurs fois le même JSX ou la même logique lorsque cela peut être factorisé.
- Les états principaux doivent être placés dans un composant approprié et les données doivent circuler clairement entre parent et enfant.
- Les noms de variables, fonctions, states, props et composants doivent être significatifs et cohérents.
- Le code inutilisé, les répétitions inutiles et les blocs devenus obsolètes doivent être retirés.
- Les commentaires doivent être utiles : expliquez une logique ou un choix qui n'est pas évident, plutôt que commenter chaque ligne.
- L'interface doit utiliser React-Bootstrap et des CSS Modules de manière cohérente.
- L'interface doit être responsive et garder une présentation cohérente sur différentes tailles d'écran.

C. Documentation et version finale de l'analyse

Aucun rapport long n'est demandé. Le dépôt doit toutefois contenir un README.md clair avec :

- votre nom et le titre du TP;
- les commandes nécessaires pour lancer l'application;
- une section « Analyse des besoins » contenant la version finale de votre liste de besoins;
- la version finale de votre arbre des composants, sous forme de schéma, d'image ou d'arbre textuel lisible;
- une courte description du rôle des principaux composants;

Programmation avancée

- une capture d'écran du résultat final;
- les sources des images ou ressources externes utilisées, s'il y a lieu.

D. GitHub et remise finale

- Le même dépôt utilisé pendant la partie 1 doit être conservé jusqu'à la remise finale.
- L'historique Git doit montrer une progression normale du travail et contenir des messages de commit significatifs.
- Le lien du dépôt doit être remis sur la plateforme du cours avant l'échéance.
- La version évaluée est la version du dépôt présente au moment de la remise finale.

Critère — Partie 2	Ce qui est évalué	Points
Fonctionnalités demandées	Les besoins du cahier des charges sont couverts et les interactions fonctionnent.	2,5
Architecture et réutilisation	Structure claire, responsabilités précises, composants réutilisables, peu de répétitions.	2,0
React : state, props, listes et événements	useState, props, map(), key, filter(), setters et affichage conditionnel sont utilisés correctement.	2,5
Interface et responsive	React-Bootstrap, CSS Modules, cohérence visuelle et adaptation aux tailles d'écran.	1,5
Qualité, documentation et Git	Nommage, lisibilité, commentaires utiles, README et historique Git cohérent.	1,5

Partie 3 — Entrevue individuelle de validation (10 %)

Modalité : l'entrevue aura lieu après la remise finale. La séance et l'heure de l'entrevue seront annoncées en classe.

Objectif : vérifier que vous comprenez le code remis, les choix réalisés et le fonctionnement de votre propre solution.

- Durée indicative : environ 6 à 8 minutes par étudiant(e).
- Vous ouvrirez le dépôt correspondant à votre remise finale et lancerez l'application.
- L'enseignante vous posera quelques questions ciblées sur l'architecture, le state, les événements, les listes, les props et le flux des données.
- Une petite modification en direct pourra être demandée afin de vérifier votre capacité à adapter le code que vous avez remis.
- L'évaluation porte sur la compréhension et la justification, et non sur la mémorisation exacte de la syntaxe.
- Sauf consigne contraire, la modification effectuée pendant l'entrevue sert à l'évaluation et ne constitue pas une nouvelle remise du TP.

Exemples de questions pouvant être posées

- Expliquez le rôle de deux composants de votre choix et la relation entre eux.
- Pourquoi avez-vous placé ce state dans ce composant plutôt que dans un autre ?
- Comment les données d'un projet arrivent-elles jusqu'au composant qui l'affiche ?
- Que fait map() dans votre solution et pourquoi avez-vous besoin d'une key ?
- Expliquez comment votre filtre produit la liste affichée.
- Montrez où une fonction est passée d'un parent vers un enfant par prop.
- Montrez un exemple d'affichage ou de style conditionnel dans votre code.
- Effectuez une petite adaptation demandée par l'enseignante.

Critère — Partie 3	Ce qui est évalué	Points
Architecture	Explique les responsabilités des composants et le flux général des données.	2,0
State et événements	Explique les states, les fonctions, les événements et l'affichage conditionnel.	2,0
Listes	Explique map(), key, filter() et la modification d'une liste.	2,0
Props	Explique les échanges parent-enfant et les fonctions passées par props.	2,0
Modification en direct	Adapte ou explique correctement une petite modification dans son propre code.	2,0

Programmation avancée

Travail à remettre

- Le lien du dépôt GitHub individuel.
- Le code final fonctionnel.
- Le README.md contenant la version finale de l'analyse des besoins, l'arbre des composants et les informations demandées.
- L'historique Git du projet, incluant le dépôt de progression réalisé pendant la séance en présentiel.

Résumé de la pondération

Partie	Modalité	Éléments principaux	Pondération
Partie 1	En classe + version finale	Analyse des besoins, arbre des composants, correction du code de départ, trace GitHub	5 %
Partie 2	Remise finale	Solution fonctionnelle, architecture, réutilisation, qualité, documentation	10 %
Partie 3	Après la remise finale	Compréhension, justification et petite modification en direct	10 %
TOTAL			25 %

Rappels importants

- Le travail est individuel.
- Le code fourni par l'enseignante doit être utilisé comme point de départ.
- Votre analyse et votre arbre des composants peuvent évoluer entre la séance en classe et la remise finale.
- Le dépôt de progression à la fin de la séance en présentiel est obligatoire.
- La qualité de la langue française est évaluée conformément aux règles du cours.